



Vijay Academy Presents



Savitribai Phule Pune University (SPPU)

Data Science and Big Data Analytics (DSBDA)

TE Computer Engineering (2019 Pattern)

One Shot Lecture

Unit 4. Predictive Big Data Analytics with Python

What You Get : Strategy For Exam :

- Simple Explanation
- PPT + Notes
- PYQs + Most IMP Questions



Subscribe The Vijay Academy For Free Engineering Content

Unit 4: Predictive Big Data Analytics with Python

Syllabus Topics:

1. Introduction to Predictive Big Data Analytics with Python
2. Essential Python Libraries
3. Data Preprocessing - Removing Duplicates
4. Data Preprocessing - Transformation of Data
5. Data Preprocessing - Replacing Values
6. Data Preprocessing - Handling Missing Data
7. Analytics Types (Predictive, Descriptive, Prescriptive)
8. Association Rules and Apriori Algorithm
9. FP Growth Algorithm
10. Linear Regression & Logistic Regression
11. Naive Bayes Classifier
12. Decision Trees
13. Introduction to Scikit-learn
14. Matplotlib



1: Introduction to Predictive Big Data Analytics with Python

What is Predictive Big Data Analytics?

Predictive Big Data Analytics is the process of **using statistical algorithms and machine learning techniques on big data** to predict future outcomes and trends.

Simply put — *using past big data to predict what will happen in the future.*

Why Python for Big Data Analytics?

Python is the **most popular programming language** for data science and big data analytics because:

1. Simple and Easy to Learn

Clean and readable syntax

Easy for beginners to understand

2. Rich Library Ecosystem

Thousands of libraries for data science

3. Open Source and Free

Completely free to use

Large community support

4. Versatile

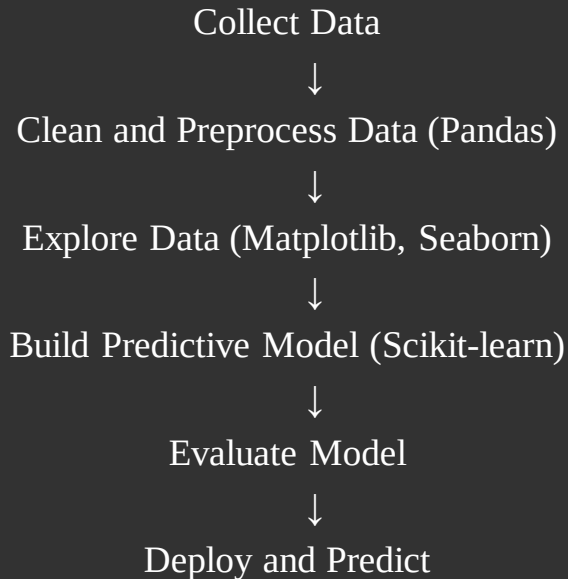
Can handle data collection, cleaning, analysis, modeling and visualization

Works for both small and big data

5. Integration with Big Data Tools

Works well with Hadoop, Spark, NoSQL databases

Predictive Analytics Process with Python:





2: Essential Python Libraries

What are Python Libraries?

Python Libraries are **pre-written collections of code** that provide ready-made functions and tools for specific tasks. Simply put — *instead of writing everything from scratch, we use libraries that already have the code ready.*

Essential Python Libraries for Data Science:

1. NumPy

Full Name: Numerical Python

What it does:

Provides support for **large multi-dimensional arrays and matrices**

Contains mathematical functions for operations on arrays

Foundation for all other data science libraries

Example :

```
import numpy as np
```

```
# Create array
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)      # [1 2 3 4 5]
```

```
# Mathematical operations
```

```
print(np.max(arr)) # 5
```

```
print(np.min(arr)) # 1
```



2. Pandas

What it does:

Provides **DataFrame** structure

(like Excel table in Python)

Used for **data manipulation and cleaning**

Most important library for data preprocessing

Example:

```
import pandas as pd
```

```
# Create DataFrame
```

```
data = {'Name': ['Ram', 'Priya', 'John'],  
        'Age': [20, 21, 22],  
        'Marks': [85, 90, 78]}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

3. Matplotlib

What it does:

Used for **creating visualizations and plots**

Creates bar charts, line charts, histograms, scatter plots

Most basic visualization library in Python

Example:

```
import matplotlib.pyplot as plt
```

```
# Bar chart
```

```
subjects = ['Maths', 'Science', 'English']  
marks = [85, 90, 78]  
plt.bar(subjects, marks)  
plt.show()
```

4. Seaborn

What it does:

Built on top of Matplotlib

Creates **more beautiful and statistical visualizations**

Easier to use than Matplotlib

Example:

```
import seaborn as sns
import pandas as pd

# Load built-in dataset
df = sns.load_dataset('iris')

# Beautiful scatter plot
sns.scatterplot(x='sepal_length',
                y='sepal_width',
                hue='species',
                data=df)
```

5. Scikit-learn

What it does:

Most popular library for **Machine Learning**

Contains algorithms for classification, regression, clustering

Tools for model evaluation and preprocessing

Example:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

# Build model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
predictions = model.predict(X_test)
```



6. SciPy

What it does:

Used for **scientific and statistical computing**

Contains functions for statistics, optimization, signal processing

Built on top of NumPy

Example:

```
from scipy import stats
```

```
# Calculate t-test
```

```
data1 = [20, 30, 40, 50]
```

```
data2 = [25, 35, 45, 55]
```

```
t_stat, p_value = stats.ttest_ind(data1, data2)
```

```
print(p_value)
```


3: Data Preprocessing - Removing Duplicates

What is Removing Duplicates?

Removing Duplicates means **identifying and deleting repeated records** that appear more than once in a dataset.

Simply put — *finding and removing same data that is entered multiple times.*

Why Remove Duplicates?

Duplicates cause **incorrect analysis results**

Example: If same sale is recorded twice, total revenue will be wrong

Wastes **storage space**

Leads to **wrong predictions** in ML models

Makes data **unreliable and inaccurate**

Methods to Identify Duplicates:

Check for **same values in all columns**

Check for **same ID or primary key**

Check for **same name and contact number**

Use **sorting** to bring duplicates together

Example of Duplicate Data:

Before Removing Duplicates:

ID	Name	Age	Marks
1	Ram	20	85
2	Priya	21	90
3	Ram	20	85
4	John	22	78
5	Priya	21	90

After Removing Duplicates:

ID	Name	Age	Marks
1	Ram	20	85
2	Priya	21	90
3	John	22	78

4: Data Preprocessing - Transformation of Data

What is Data Transformation?

Data Transformation is the process of **converting data from one format, structure or scale into another** to make it suitable for analysis.

Simply put — *changing the form of data to make it more useful and consistent.*

Why Do We Need Data Transformation?

Raw data is often in **different scales and formats**

ML models work better with **normalized and consistent data**

Helps in **removing inconsistencies**

Makes data **comparable across different attributes**



Methods of Data Transformation:

Method	Purpose	Example
Normalization	Scale to 0-1	Age, Salary
Standardization	Mean=0, SD=1	Height, Weight
Function/Mapping	Apply math function	Log of income
Aggregation	Summarize data	Daily→Monthly
Generalization	Low→High level	City→Country
Encoding	Text→Numbers	Male=0, Female=1

Example of Transformation:

Before Transformation:

Name	Gender	Age	Salary
Ram	Male	25	50000
Priya	Female	30	80000
John	Male	35	120000

After Transformation:

Name	Gender	Age (Normalized)	Salary (Log)
Ram	0	0.0	10.82
Priya	1	0.5	11.29
John	0	1.0	11.70

5: Data Preprocessing - Replacing Values

What is Replacing Values?

Replacing Values means **identifying incorrect, inconsistent or unwanted values** in a dataset and **replacing them with correct or appropriate values**.

Simply put — *finding wrong or bad values in data and fixing them by replacing with better values.*

Why Do We Need to Replace Values?

Data collected from multiple sources has **different formats values**

Human errors cause **wrong entries** in data

Some values are old or expired **or irrelevant**

Incorrect values lead to **wrong analysis and predictions**

Methods of Replacing Values:

1. Replace with Constant Value

Replace wrong value with a **fixed constant**

Example: Replace all negative ages with 0

2. Replace with Mean/Median/Mode

Replace wrong value with **statistical measure**

Example: Replace **extreme** salary with mean salary.

3. Replace with Previous or Next Value

Replace missing value with **neighboring value**

Example: In time series, replace missing day's sales with previous day's sales

Example:

Before Replacing Values:

ID	Gender	Age	City
1	M	25	mumbai
2	Female	-5	Mumbai
3	MALE	30	MUMBAI
4	f	25	pune

ID	Gender	Age	City
1	Male	25	Mumbai
2	Female	25	Mumbai
3	Male	30	Mumbai
4	Female	25	Pune

6: Data Preprocessing - Handling Missing Data

What is Missing Data?

Missing Data means **values that are absent or not recorded** in a dataset.

Simply put — *empty or blank cells in your data table.*

Why Does Missing Data Occur?

Human error while entering data manually

Data **not collected** at time of entry

System failures during data collection

Respondents **skipping questions** in surveys

Data **lost during transfer** from one system to another

Methods to Handle Missing Data :

Method	Explanation
Delete Record	Simply remove rows that have missing values
Fill with Mean	Replace missing number with average of that column
Fill with Median	Replace missing number with middle value of that column
Fill with Mode	Replace missing value with most frequently occurring value
Fill with Constant	Replace missing value with a fixed value like 0 or "Unknown"
Forward Fill	Replace missing value with previous row's value
Backward Fill	Replace missing value with next row's value
ML Prediction	Use machine learning model to predict and fill missing value

Example :

ID	Name	Age	Salary	City
1	Ram	25	50000	Mumbai
2	Priya	?	80000	Pune
3	John	30	?	Mumbai
4	Sara	?	60000	?
5	Raj	28	70000	Delhi

Final Dataset After Handling Missing Values:

ID	Name	Age	Salary	City
1	Ram	25	50000	Mumbai
2	Priya	28	80000	Pune
3	John	30	65000	Mumbai
4	Sara	28	60000	Mumbai
5	Raj	28	70000	Delhi

Missing **Age** (numerical, normal) → filled with **Mean = 28**

Missing **Salary** (numerical, has outliers) → filled with **Median = 65000**

Missing **City** (categorical) → filled with **Mode = Mumbai**



7. Types of Analytics :

What are Types of Analytics?

Analytics in Big Data refers to **different ways of analyzing data** to get insights and make decisions.

There are **4 types** of analytics:

1. Descriptive Analytics

Describes **what happened** in the past using historical data.

Simply put — *looking at past data to understand what occurred.*

Question answered: "What happened?"

Techniques: Summary statistics, reports, dashboards

Tools: Tableau, Power BI, Excel

Example: Monthly sales report showing total revenue last year

2. Diagnostic Analytics

Explains **why something happened** by finding causes and relationships.

Simply put — *digging deeper to find the reason behind what happened.*

Question answered: "Why did it happen?"

Techniques: Drill down analysis, correlation, data mining

Tools: SQL, Python, R

Example: Finding why sales dropped in December by analyzing customer complaints and market trends

3. Predictive Analytics

Predicts **what will happen** in the future using statistical models and ML.

Simply put — *using past data to forecast future outcomes.*

Question answered: "What will happen?"

Techniques: Regression, Decision Trees, ML algorithms

Tools: Python Scikit-learn, SAS, R

Example: Predicting which customers will leave next month

4. Prescriptive Analytics

Recommends **what action to take** to achieve desired outcome.

Simply put — *not just predicting future but also suggesting best action.*

Question answered: "What should we do?"

Techniques: Optimization, simulation, AI

Tools: IBM SPSS, SAS, Python

Example: Recommending which customers to target with discount offers to prevent churn

8: Association Rules and Apriori Algorithm

What are Association Rules?

Association Rules are **if-then statements** that help discover **interesting relationships and patterns** between variables in large datasets.

Simply put — *finding which items are frequently bought or used together.*

Classic Example:

People who buy **bread** also buy **butter**

This is called **Market Basket Analysis**

Key Terms in Association Rules:

1. Itemset

A **collection of items** in a transaction.

Example: {Bread, Butter, Milk} is an itemset.



2. Support

Tells how **frequently an itemset appears** in dataset

Formula:

$$\text{Support(A)} = \frac{\text{Number of transactions containing A}}{\text{Total number of transactions}}$$

Example: Bread appears in 3 out of 5 transactions

$$\text{Support(Bread)} = 3/5 = 0.6 = 60\%$$

3. Confidence

Tells how often the rule is **correct**.

Formula:

$$\text{Confidence(A} \rightarrow \text{B)} = \frac{\text{Support(A and B)}}{\text{Support(A)}}$$

Example:

Imagine 5 friends went shopping:

Friend	Bought Bread?	Bought Butter?
Ram	Yes	Yes
Priya	Yes	Yes
John	Yes	No
Sara	No	Yes
Raj	No	No

Confidence (Bread $\rightarrow$ Butter):

Question: Out of people who bought Bread, how many also bought Butter?

People who bought Bread = **Ram, Priya, John = 3 people**
Out of these 3, who also bought Butter = **Ram, Priya = 2 people**

$$\text{Confidence} = 2/3 = 0.67 = 67\%$$

4. Lift

Tells how **strong the association** is between items.

Formula:

$$\text{Lift}(A \rightarrow B) = \text{Confidence}(A \rightarrow B) \div \text{Support}(B)$$

Lift > 1 → **Positive association**

Lift = 1 → **No association**

Lift < 1 → **Negative association**

Apriori Algorithm?

What is Apriori Algorithm?

Apriori Algorithm is a **classic algorithm** used to find **frequent itemsets** and generate **association rules** from transaction data.

Simply put — *finding which items are bought together most frequently.*



Steps of Apriori Algorithm:

Step 1: Set Minimum Support and Confidence

Define **minimum support threshold**

Define **minimum confidence threshold**

Example: Min Support = 50%, Min Confidence = 60%

Step 2: Find Frequent 1-Itemsets (L1)

Count **frequency of each individual item**

Keep only items that meet **minimum support**

Step 3: Generate Candidate 2-Itemsets (C2)

Combine frequent 1-itemsets to create **pairs**

Calculate support for each pair

Step 4: Find Frequent 2-Itemsets (L2)

Keep only pairs that meet **minimum support**

Remove pairs below minimum support

Step 5: Repeat Until No More Frequent Itemsets

Keep generating larger itemsets

Stop when no more frequent itemsets found

Step 6: Generate Association Rules

From frequent itemsets generate **association rules**

Keep only rules that meet **minimum confidence**

9: FP Growth Algorithm

What is FP Growth Algorithm?

FP Growth stands for **Frequent Pattern Growth**. It is an **improved and faster algorithm** for finding frequent itemsets compared to Apriori Algorithm.

Simply put — *a better and faster way to find which items are frequently bought together without generating too many candidate itemsets.*

Why FP Growth over Apriori?

Apriori is **slow** because it scans database multiple times

Apriori generates **too many candidate itemsets**

FP Growth scans database **only twice**

FP Growth uses a special **tree structure** called FP Tree

Much **faster and memory efficient** than Apriori

9: FP Growth Algorithm

Steps of FP Growth Algorithm:

Step 1: Scan Database First Time

Count **frequency of each item**

Remove items below **minimum support**

Sort remaining items in **decreasing order of frequency**

Step 2: Scan Database Second Time

Build **FP Tree** by inserting each transaction

Items in each transaction are **sorted by frequency**

Step 3: Mine FP Tree

Extract **frequent patterns** from FP Tree

Generate **association rules** from frequent patterns.

10: Linear Regression

What is Linear Regression?

Linear Regression is a **statistical method** used to find the **relationship between two or more variables** and predict a **continuous numerical value**.

Simply put — *drawing a best fit straight line through data points to predict future values.*

Primary Objectives of Linear Regression:

Find **relationship** between variables

Predict future values

Understand **impact** of one variable on another

Find **best fit line** that minimizes errors



Types of Linear Regression:

1. Simple Linear Regression

Relationship between **one input variable (X)** and **one output variable (Y)**

Formula:

$$Y = mX + c$$

Where:

Y = Predicted value (dependent variable)

X = Input value (independent variable)

m = Slope of line

c = Intercept (where line crosses Y axis)

2. Multiple Linear Regression

Relationship between **multiple input variables** and **one output variable**

Formula:

$$Y = m_1X_1 + m_2X_2 + m_3X_3 + \dots + c$$

Where:

X1, X2, X3 = Multiple input variables

m1, m2, m3 = Coefficients

c = Intercept

11: Logistic Regression

What is Logistic Regression?

Logistic Regression is a **classification algorithm** used to predict **categorical outcomes** (yes/no, spam/not spam, 0/1).
Simply put — *predicting which category or class a data point belongs to.*

Sigmoid Function

Logistic Regression uses **Sigmoid Function** to convert any value into a probability between 0 and 1.

Formula:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$

Where:

$z = m_1X_1 + m_2X_2 + \dots + c$ (linear combination)

e = Euler's number (2.718)

Properties of Sigmoid:

Output always between **0 and 1**

When z is very large $\rightarrow$ output close to **1**

When z is very small $\rightarrow$ output close to **0**

When $z = 0 \rightarrow$ output = **0.5**

11: Logistic Regression

How Logistic Regression Works:

Step 1: Calculate Linear Combination

$$z = m_1X_1 + m_2X_2 + c$$

Step 2: Apply Sigmoid Function

$$\text{Probability} = 1 / (1 + e^{(-z)})$$

Step 3: Apply Threshold

If Probability $\geq 0.5 \rightarrow$ Class 1 (Yes/Spam/Positive)

If Probability $< 0.5 \rightarrow$ Class 0 (No/Not Spam/Negative)

12: Naive Bayes Classifier

What is Naive Bayes Classifier?

Naive Bayes is a **classification algorithm** based on **Bayes Theorem** that predicts the class of a data point based on probability.

Simply put — *using probability to classify data into categories.*

How Naive Bayes Works:

Uses **Bayes Theorem** to calculate probability of each class and picks the class with **highest probability**.

$$P(\text{Class}|\text{Features}) = P(\text{Features}|\text{Class}) \times P(\text{Class}) \div P(\text{Features})$$

Where:

$P(\text{Class}|\text{Features})$ = Probability of class given features

$P(\text{Features}|\text{Class})$ = Probability of features given class

$P(\text{Class})$ = Prior probability of class

$P(\text{Features})$ = Overall probability of features

12: Naive Bayes Classifier



Advantages of Naive Bayes:

Very **fast and simple**

Works well with **small datasets**

Handles **multiple classes** easily

Works great for **text classification**

Less training data required

Disadvantages of Naive Bayes:

Cannot handle **feature correlations** well

Poor performance with **insufficient data**

Applications of Naive Bayes:

Email spam detection

Sentiment analysis (positive/negative review)

Medical diagnosis (disease or no disease)

News classification (sports, politics, entertainment)

Document categorization

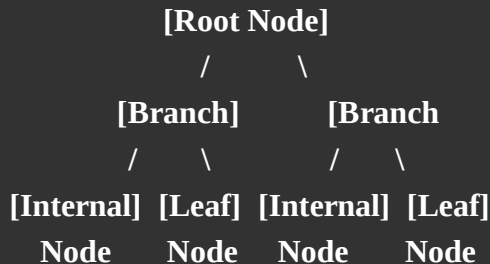
13: Decision Trees

What is a Decision Tree?

A Decision Tree is a **tree shaped model** that makes decisions by splitting data into branches based on feature values until it reaches a final decision.

Simply put — *a flowchart like structure that helps make decisions step by step like a tree.*

Structure of Decision Tree:



Parts of Decision Tree:

Part	Description	Example
Root Node	Starting point, first split	Is Age > 30?
Internal Node	Intermediate decision point	Is Income > 50000?
Branch	Outcome of a decision	Yes or No
Leaf Node	Final decision/class	Approved or Rejected

Key Concepts:

1. Entropy

Entropy measures **impurity or disorder** in a dataset.

Simply put — *how mixed up the data is.*

Formula:

$$\text{Entropy} = -\sum P(i) \times \log_2(P(i))$$

Where $P(i)$ = probability of class i

Values:

Entropy = 0 → **Pure node** (all same class)

Entropy = 1 → **Maximum impurity** (equally mixed)

2. Information Gain

Information Gain measures **how much a feature reduces entropy** after splitting.

Simply put — *how useful a feature is for making decisions.*

Formula:

$$\text{Information Gain} = \text{Entropy}(\text{Parent}) - \text{Weighted Average Entropy}(\text{Children})$$

Higher Information Gain → Better feature for splitting



14: Introduction to Scikit-learn

What is Scikit-learn?

Scikit-learn is a **free and open source Machine Learning library** for Python that provides simple and efficient tools for data mining and data analysis.

Simply put — *a ready made toolbox for building machine learning models in Python.*

Key Features of Scikit-learn:

Built on **NumPy, SciPy and Matplotlib**

Contains **all major ML algorithms**

Provides tools for **preprocessing, model building and evaluation**

Simple and consistent API

Free and open source

Excellent **documentation and community support**

Installation of Scikit-learn:

```
pip install scikit-learn
```

14: Introduction to Scikit-learn

Datasets Available in Scikit-learn:

Scikit-learn provides **built-in datasets** for practice:

Dataset	Description	Use
Iris	Flower measurements	Classification
Boston Housing	House prices	Regression
Digits	Handwritten digits	Classification
Wine	Wine characteristics	Classification
Breast Cancer	Cancer data	Classification

Advantages of Scikit-learn:

Easy to use with consistent API

Contains **all major ML algorithms**

Excellent **documentation**

Works well with **NumPy and Pandas**

Free and open source



15: Matplotlib

What is Matplotlib?

Matplotlib is a **data visualization library** in Python used to create **charts, graphs and plots** from data.

Simply put — *a tool to draw graphs and charts in Python to understand data visually.*

Why Use Matplotlib?

Makes data **easy to understand visually**

Helps find **patterns and trends** in data

Used for **Exploratory Data Analysis (EDA)**

Creates **quality** plots

Free and open source

Installation:

```
pip install matplotlib
```

Basic Import:

```
import matplotlib.pyplot as plt
```

Plot Types:

Plot	Used For	Example
Line Plot	Trends over time	Sales over months
Bar Chart	Compare categories	Subject marks
Histogram	Data distribution	Marks distribution
Scatter Plot	Relationship between variables	Hours vs Marks
Pie Chart	Proportions/percentages	Market share
Box Plot	Distribution and outliers	Salary range



Thank You!



Like, Share & Subscribe

<https://youtube.com/@thevijayacademy?si=uYwlDuAxyZfLsiei>

Also join Telegram channel for updates and notes :

https://t.me/the_vijay_academy